

TP 6/7 : algorithmes de tri

Exercice 1 : fonction de génération de tableaux

Ecrire la fonction `T = Genere_Tableau(n)` qui génère un tableau de `n` valeurs entières aléatoires entre 0 et 100 000.

Exercice 2 : fonction d'insertion d'élément

Programmez la fonction GNU Octave `T = Insérer_Element(T, p_init, p_fin)` qui réalise l'insertion d'un élément à partir d'un tableau de nombre entiers, d'un indice de position initiale (`p_init`) et d'un indice de position finale (`p_fin`).

Exercice 3 : tri par insertion

Le tri par insertion consiste à classer les deux premiers éléments du tableau. Une fois que les deux premiers sont ordonnés, on prend l'élément qui suit et on le classe à son tour dans ce qui a déjà été classé. Pour chaque élément `i` du tableau, on sait que les éléments de 1 à `i-1` sont déjà classés. On va chercher la position `j` parmi les `i-1` premiers éléments de manière à ce que `T[i] < T[j]`. On insère alors `T[i]` à la position `j`.

Exercice 4 : tri par sélection

Le tri par sélection consiste à chercher le plus petit élément du tableau et à le placer en première position. Une fois le plus petit élément positionné en première position, on recommence la même opération en commençant à l'indice 2 : on cherche le plus petit élément compris dans le tableau entre la position 2 et la fin du tableau, puis on place cet élément à la position 2. Puis on recommence ainsi en partant de l'élément 3, 4 et ainsi de suite jusqu'au bout du tableau.

Exercice 5 : tri à bulles (Bubble sort)

Ecrivez la fonction `tri_a_bulles` définie par l'algorithme suivant

```
fonction t= tri_a_bulles(tableau t)
    taille = taille(t);
    pour i de taille à 1
        pour j de 2 à i
            si t[j-1]>t[j] alors
                échanger t[j-1] et t[j]
            finsi
        finpour
    finpour
fin
```

Exercice 6 : tri rapide (QuickSort)

Ecrivez les fonctions `partitionner`, `tri_rapide` et `tri_r` définies par les algorithmes suivants :

```
fonction [p, t] = partitionner(tableau t, premier, dernier)
    pivot = t[premier] ;
    p = premier ;
    pour i de premier+1 à dernier
        si t[i] <= pivot alors
            p = p+1 ;
            échanger t[i] et t[p]
        finsi
    finpour
    échanger t[premier] et t[p]
fin

fonction t = tri_rapide(tableau t, premier, dernier)
    début
        si premier < dernier alors
            [pivot, t] = partitionner(t, premier, dernier)
            t = tri_rapide(t,premier,pivot-1)
            t= tri_rapide(t,pivot+1,dernier)
        fin si
    fin

fonction t= tri_quicksort(tableau t)
    taille = taille(t);
    t = tri_rapide(t, 1, taille);
fin
```

Exercice 7 : comparaison des quatre méthodes de tri

Les trois méthodes de tri seront évaluées au moyen de deux variables : **le nombre d'éléments à trier** et **le temps nécessaire** pour réorganiser le tableau (utilisez par exemple les fonctions GNU Octave `tic()` et `toc()`).

Il vous est demandé d'évaluer ces quatre méthodes de tri pour des tableaux de taille de **50 à 1 050** éléments aléatoires en augmentant à chaque itération la taille du tableau de 100 éléments.

A chaque fois, vous devrez créer un tableau de la taille demandée avec des valeurs générées aléatoirement. Ce tableau devra alors être trié successivement avec les quatre méthodes de tri. Pour chaque taille de tableau, les tris seront évalués **10 fois** chacun afin de voir si le temps d'exécution varie selon les éléments.

Vous générerez aussi un tableau déjà trié et un tableau trié de manière inverse et effectuerez les mêmes opérations que précédemment.

Les résultats d'évaluation seront présentés :

- **sous la forme d'un tableau** pour les résultats concernant le temps d'exécution. Ce tableau aura la forme suivante :

<i>Taille</i>	<i>Temps moyen insertion</i>	<i>Temps moyen sélection</i>	<i>Temps moyen tri à bulles</i>	<i>Temps moyen QuickSort</i>
---------------	----------------------------------	----------------------------------	-------------------------------------	----------------------------------

Les temps moyens seront calculés sur les 10 essais effectués pour chaque tri et chaque taille.

- **sous la forme d'un graphique 2D** représentant le temps nécessaire pour chaque méthode de tri.
Le graphique aura comme abscisse le nombre d'éléments du tableau à trier, en ordonnée la variable étudiée (temps) et chaque méthode de tri sera représentée par une courbe de couleur différente.

Liens :

- **Les tris sous forme de danse folklorique :**
<https://www.laboiteverte.fr/algorithmes-tri-visualises-danses-folkloriques>
- **Les algorithmes de tris :** <https://interstices.info/les-algorithmes-de-tri>